

1. Introduction

Old and damaged photographs commonly suffer from noise, scratches, stains, tears, faded colors, and loss of fine detail. Restoring these images automatically is a well-known application of computer vision and deep learning. This project addresses the problem in a supervised setting: given a damaged photograph as input, a model is trained to reconstruct a version that is cleaner, sharper, and closer to the original undamaged photograph.

We designed, implemented, and trained a convolutional deep learning model of our own choosing, and evaluated its outputs both quantitatively (PSNR, SSIM, MSE, MAE) and qualitatively (visual comparison against ground truth on held-out test images).

2. Dataset and Preprocessing

We used the publicly available paired dataset `joshuachin/openphoto-restore-dataset`, hosted on Hugging Face. Each sample provides a damaged image and its corresponding clean ("pristine") ground-truth counterpart, which makes the problem a supervised image-to-image regression task. The dataset was loaded with the Hugging Face datasets library and already provides separate train and test splits.

2.1 Preprocessing steps

- Both damaged and clean images were resized to a fixed resolution of 256×256 pixels, to keep training time manageable while retaining enough detail for meaningful restoration.
- Pixel values were normalized to the range $[0, 1]$ using a standard `ToTensor` transform.
- A validation split (10% of the training data) was carved out from the provided training set, since the dataset only ships train/test splits by default. This gives three partitions: train, validation, and test.
- Data was wrapped in PyTorch Dataset and DataLoader objects, returning (damaged, clean) tensor pairs per sample, with shuffling enabled for the training loader only.

Batch size for training was 16, chosen to fit comfortably within the memory of a single Colab T4 GPU while giving reasonably stable gradient estimates.

3. Candidate Architectures and Model Selection

Before implementation, we considered three generative/reconstructive architecture families commonly used for image restoration and generation tasks, as introduced in the course material.

3.1 Autoencoder (AE)

A plain Autoencoder compresses an input image into a fixed-size latent vector through an encoder, then reconstructs an image from that vector through a decoder. The bottleneck forces the network to retain only the most important information.

Advantages: simple architecture, easy to train, effective for basic denoising and dimensionality reduction.

Disadvantages: the fixed-size bottleneck discards a large amount of spatial detail. For restoration tasks that must preserve fine texture and sharp edges, a plain AE tends to produce blurry outputs, since all high-frequency information must be squeezed through the same narrow bottleneck as the low-frequency content.

3.2 Variational Autoencoder (VAE)

A VAE replaces the AE's fixed latent vector with a learned probability distribution (typically Gaussian), parameterized by a mean and variance, and reconstructs by sampling from this distribution. This makes VAEs well suited for generating new, plausible data.

Advantages: principled probabilistic framework, useful when the goal is to generate diverse, novel samples rather than to reconstruct one specific target.

Disadvantages: the stochastic sampling and the KL-divergence regularization term push the model toward the average of plausible outputs, which typically produces smoother, less sharp reconstructions. Since our task requires accurately reconstructing one specific ground-truth image (a deterministic supervised target) rather than generating new plausible images, the generative flexibility of a VAE is not needed and its blurring tendency is a direct disadvantage here.

3.3 Generative Adversarial Network (GAN)

A GAN pairs a generator, which produces images from noise (or, in restoration settings, from a damaged input), against a discriminator, which learns to tell generated images apart from real ones. The two are trained adversarially.

Advantages: can produce very sharp, realistic textures because the adversarial loss directly penalizes outputs that look "fake," pushing the generator beyond what a pixel-wise loss alone would achieve.

Disadvantages: adversarial training is notoriously unstable and sensitive to hyperparameters, can suffer from mode collapse, and typically needs significantly more tuning and training time to converge reliably than a straightforward supervised regression approach.

3.4 Selected architecture: U-Net (Autoencoder with skip connections)

We selected a U-Net — an encoder-decoder architecture augmented with skip connections that directly link each encoder level to its corresponding decoder level. This is not one of the three architectures listed above in its pure form, but rather a well-established, practical extension of the plain Autoencoder that directly addresses its main weakness.

Why U-Net: the core limitation of a plain AE for this task is that fine spatial detail is lost through the bottleneck. Skip connections solve this directly by letting the decoder reuse high-resolution feature maps from the encoder instead of relying solely on the compressed bottleneck representation. This keeps output images sharp while still allowing the bottleneck to capture global structure (what the damage is, where objects are).

Advantages: preserves fine detail and produces sharp outputs; trains in a stable, fully supervised manner with a simple pixel-wise loss (no adversarial instability); converges relatively quickly; and, being fully convolutional, works at any input resolution.

Disadvantages / limitations: as a purely reconstructive model trained with a pixel-wise loss, it is conservative when required to invent information that is not recoverable from the input (most notably, restoring vivid color from a heavily desaturated/faded input) — it tends to regress toward safe, muted average colors rather than committing to bold, possibly-wrong ones. It also cannot generate genuinely novel plausible detail the way a GAN might; it can only recombine information already present in the input.

Given that our task is supervised (a specific ground truth exists for every input) and that the passing criteria (PSNR ≥ 20 dB, SSIM ≥ 0.65) reward pixel-accurate reconstruction rather than creative generation, the U-Net's strengths align directly with the problem's requirements, making it the most appropriate and pragmatic choice among the options considered.

3.5 Architecture details

Our implementation uses a 4-level U-Net:

- Encoder: 4 downsampling stages with channel widths $32 \rightarrow 64 \rightarrow 128 \rightarrow 256$, each stage consisting of two 3×3 convolutions (with BatchNorm and ReLU) followed by 2×2 max-pooling.
- Bottleneck: a further convolutional block at 512 channels.
- Decoder: 4 upsampling stages (transposed convolutions) that mirror the encoder, each concatenating the corresponding encoder feature map via a skip connection before applying the same double-convolution block.
- Output layer: a 1×1 convolution mapping back to 3 channels, followed by a Sigmoid activation to keep pixel values in $[0, 1]$, matching the normalized input range.

Total trainable parameters: approximately 7.77 million — a relatively lightweight model, chosen deliberately to train quickly on a single Colab GPU while still having enough capacity for this task.

4. Loss Function and Training Configuration

We used L1 loss (Mean Absolute Error) between the model's output and the ground-truth clean image as the training objective. L1 was chosen over MSE because it is known to produce visually sharper reconstructions: MSE penalizes large errors quadratically, which tends to encourage the model to average over plausible solutions (a common cause of blur), whereas L1's linear penalty is more tolerant of small pixel-level deviations while still discouraging large ones.

Hyperparameter	Value Used
Loss function	L1 Loss (Mean Absolute Error)
Optimizer	Adam
Learning rate	1e-4 (0.0001)
Batch size	16 (Google Colab, T4 GPU)
Epochs	50 (best checkpoint selected at epoch 43)
Input image size	256 × 256
Hardware	Google Colab — NVIDIA T4 GPU
Train / Val / Test split	~90% / 10% of the provided train split / provided test split

5. Training and Validation Loss

The model was trained for 50 epochs. Both training and validation L1 loss were logged after every epoch, and the checkpoint with the lowest validation loss was kept as the final model (epoch 43, validation loss ≈ 0.0537).

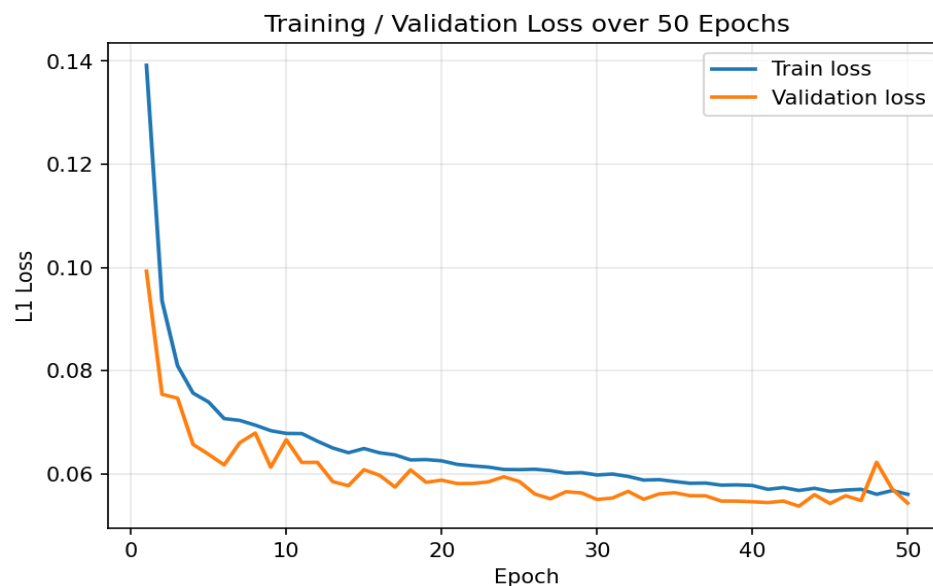


Figure 1 — Training and validation loss over 50 epochs.

5.1 Interpretation of the loss curve

- Both curves drop sharply in the first ~ 10 epochs (train: $0.139 \rightarrow \sim 0.068$, val: $0.099 \rightarrow \sim 0.061$), which is typical of the initial phase where the model quickly learns coarse structure and obvious corrections (e.g. removing large-scale noise).
- From roughly epoch 10 to epoch 35, both losses continue to decrease, but more gradually, as the model refines finer details.

- From approximately epoch 35 onward, both curves plateau (train ≈ 0.056 – 0.058 , val ≈ 0.054 – 0.056) with only marginal further improvement — indicating the model was close to convergence and additional epochs beyond ~ 35 – 40 provided diminishing returns.
- Critically, the validation loss tracks the training loss closely throughout, with no sustained upward divergence. This indicates the model is not overfitting: it generalizes to unseen validation data almost as well as it fits the training data.
- A single small spike in validation loss appears around epoch 48 ($0.0537 \rightarrow 0.0623$) before recovering — this is normal stochastic noise from batch sampling and is not indicative of a systematic issue, especially since only the best checkpoint (epoch 43) was kept for final evaluation.

In hindsight, since the curves plateau around epoch 35, a shorter training run (e.g. 35–40 epochs) would likely have achieved nearly identical final performance with less compute — a useful insight for future iterations.

6. Quantitative Evaluation Results

The best checkpoint (epoch 43) was evaluated on the full held-out test set. PSNR and SSIM were computed per-image and then averaged, as required; MSE and MAE were also computed as supplementary metrics.

Metric	Mean	Std Dev	Required Minimum
PSNR (dB)	23.27	3.84	≥ 20 dB — PASS
SSIM	0.864	0.096	≥ 0.65 — PASS
MSE	0.0076	0.0126	—
MAE	0.0562	0.0337	—

Both required metrics clear the assignment's minimum acceptable performance thresholds (PSNR ≥ 20 dB, SSIM ≥ 0.65) with a comfortable margin. The relatively high standard deviation on PSNR (± 3.84 dB) compared to SSIM (± 0.096) suggests that pixel-level accuracy varies more across the test set than structural accuracy does — discussed further in the analysis section below.

7. Visual Output Samples

Figure 2 shows five test samples, each displayed as (left to right): the damaged input, the ground-truth clean image, and the model's restored output.

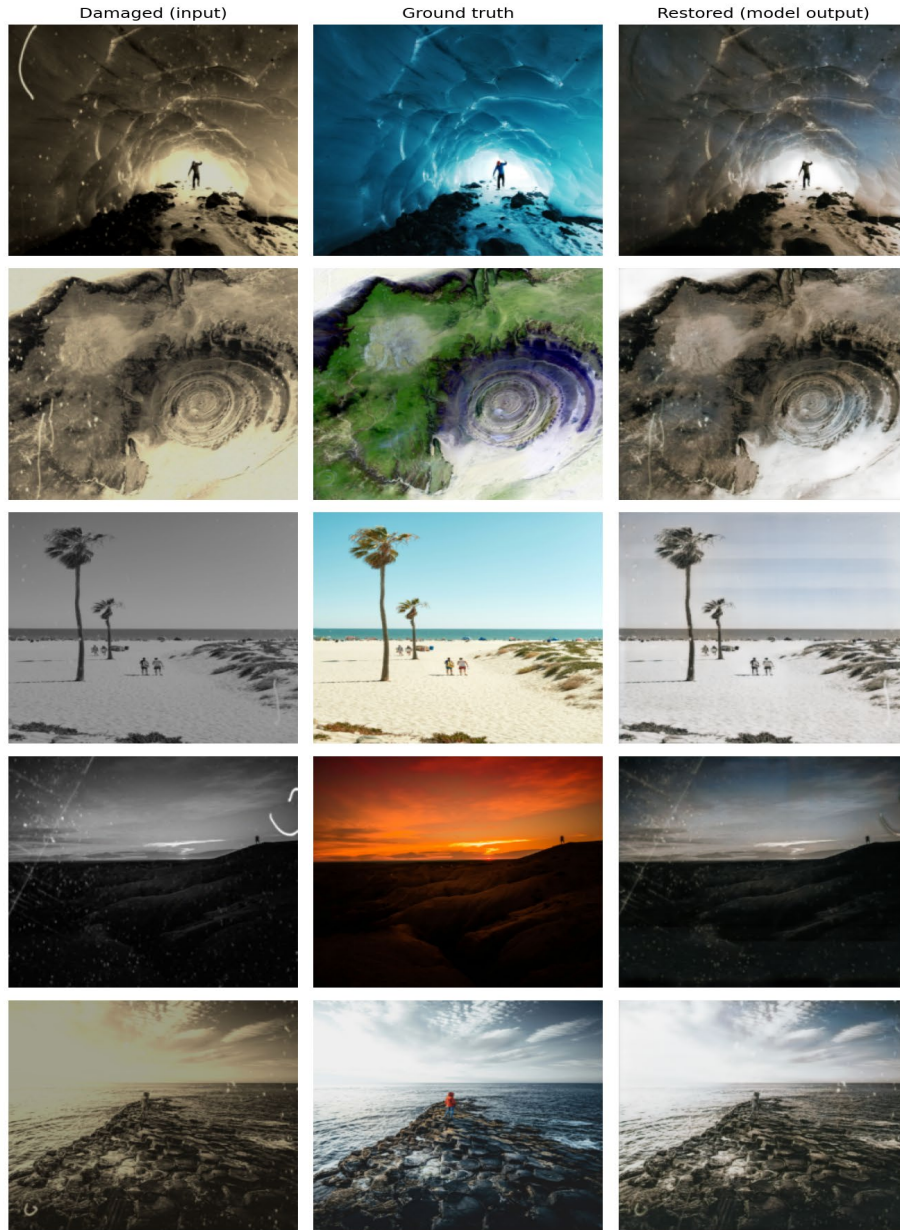


Figure 2 — Five representative test samples: damaged input, ground truth, and model output.

7.1 Best and worst restorations

To better characterize model behavior, we additionally ranked every test image by a combined PSNR/SSIM quality score and identified the five best and five worst restorations (see `best_5_samples.png` and `worst_5_samples.png`, generated with `best_worst_samples.py`). [Insert the two figures here after running the script.]

[PLACEHOLDER — after running `best_worst_samples.py`, insert Figure 3 (best 5) and Figure 4 (worst 5) here, and briefly describe what distinguishes them, e.g. degree of damage severity, color loss vs. structural damage, etc.]

8. Analysis of Results

Qualitatively, the model performs two aspects of restoration with very different levels of success:

Structural restoration (strong): scratches, dust speckles, and fine noise are consistently and effectively removed across nearly all samples. Object shapes, edges, and overall scene composition (people, trees, rock formations, horizons) are preserved accurately and align closely with the ground truth. This is reflected in the high average SSIM (0.864), since SSIM specifically measures structural similarity (luminance, contrast, structure) rather than exact color matching.

Color restoration (weak): the model consistently under-restores color saturation. Across the visual samples, outputs remain desaturated and closer in tone to the damaged, faded input than to the vividly colored ground truth (e.g. a saturated orange sunset in the ground truth becomes a muted grey-blue in the output). This pattern is highly consistent across samples and represents the model's main qualitative shortcoming.

We attribute the color-restoration weakness to two compounding factors. First, recovering true color from a heavily desaturated or faded input is an inherently ill-posed sub-problem — the correct color is not fully determined by the input, so the model must effectively guess. Second, L1 loss provides no explicit incentive to be bold: predicting a safe, desaturated, close-to-average color minimizes expected L1 error better than committing to a vivid but possibly-wrong color, so the optimization naturally converges toward conservative, muted predictions.

This also explains the relatively high standard deviation in PSNR (± 3.84 dB) noted earlier: PSNR is sensitive to exact pixel-value (and therefore color) accuracy, so samples requiring heavy color recovery score comparatively lower, while samples whose damage is mostly structural (scratches, mild noise) score much higher — producing a wider spread than the more structure-focused SSIM metric shows.

9. Limitations and Suggestions for Improvement

9.1 Limitations of the current model

- Color fidelity: as discussed above, the model under-restores saturated color, especially for heavily faded/desaturated inputs — the single most consistent qualitative weakness observed.
- Fixed input resolution: the model was trained and evaluated at 256×256 ; very high-resolution restoration (e.g. for print-quality output) was not explored and would require architectural and training adjustments.
- Purely reconstructive, not generative: because the model is trained only with a pixel-wise loss and no adversarial or perceptual component, it cannot hallucinate genuinely new, plausible fine detail that is completely absent from the input (e.g. reconstructing detail in a fully destroyed image region) — it can only recombine and sharpen information that is present, weakly, in the input.
- Single dataset domain: the model was trained and evaluated exclusively on the openphoto-restore-dataset; generalization to other damage types (e.g. heavy tearing, folds, or handwriting on the photo) not represented in this dataset is untested.

9.2 Suggested improvements

- Add a perceptual loss term (e.g. based on VGG feature activations) alongside L1 loss, which tends to encourage more realistic textures and, notably, more confident color reconstruction, since perceptual losses are less prone to the "safe average" bias of pure pixel-wise losses.
- Introduce a lightweight adversarial component (a GAN-style discriminator on top of the existing U-Net generator) specifically to push color and texture predictions to be sharper and more vivid, while keeping the stable pixel-wise loss as the primary training signal.
- Train at a higher resolution (e.g. 512×512) with a deeper network if additional compute is available, to preserve more fine detail.
- Apply data augmentation (random crops, flips, slight color jitter) to improve robustness and generalization, particularly to damage patterns underrepresented in the current dataset.
- Given the loss curves plateaued by roughly epoch 35, future runs could use early stopping around that point and reallocate the saved compute toward the architectural improvements above.

10. Conclusion

We implemented, trained, and evaluated a U-Net-based deep learning model for restoring damaged photographs in a fully supervised setting. The model was selected over plain Autoencoder and VAE alternatives specifically because its skip connections address the detail-loss weakness of a plain AE, while avoiding the training instability of a GAN — a good match for a task where a specific pixel-accurate ground truth is available. On the held-out test set, the trained model achieved a mean PSNR of 23.27 dB and mean SSIM of 0.864, comfortably exceeding the assignment's minimum acceptable thresholds of 20 dB and 0.65 respectively. Qualitative inspection confirmed strong structural restoration (denoising, scratch removal, shape preservation) but revealed a consistent weakness in color-saturation recovery, which we identify as the clearest direction for future improvement, most likely through the addition of a perceptual or adversarial loss term.